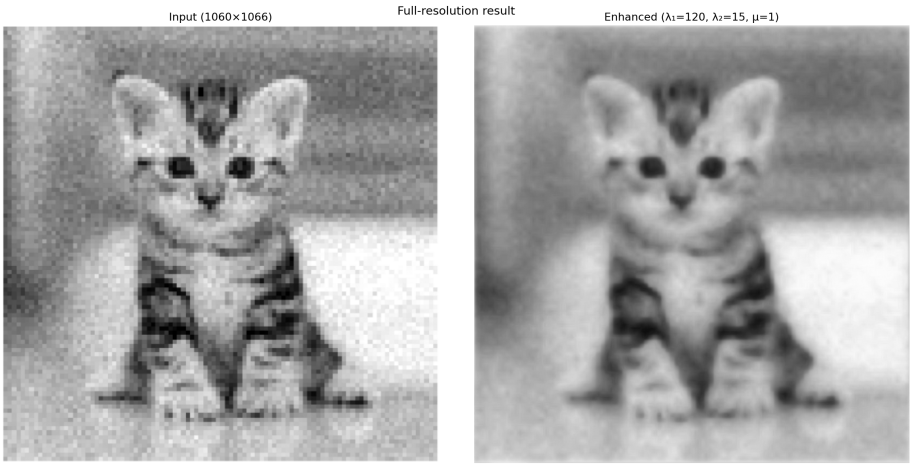
Sharpening and Smoothing a Blurry Image using Convex Optimization

Removing Noise and Blur from an Image using Total Variation and Tikhonov Regularization

Reynaldi Aditama, Nam Bui
David Ou



March – April 2026

Contents

1 Introduction **2**

2 Problem Statement **2**
2.1 Decision Variables 2
2.2 Objective Function 3
2.3 Constraints 4

3 Mathematical Tools and Theory **4**
3.1 Total Variation Norm 4
3.2 Tikhonov Regularization 5
3.3 Fast Fourier Transform (FFT) for Convolution 5
3.4 Alternating Direction Method of Multipliers (ADMM) 5
3.5 L-BFGS-B for Kernel Estimation 5

4 Why a Direct Solver is Infeasible **6**

5 Handling the Bilinear Objective: Alternating Minimization **6**

6 Modelling Process **7**
6.1 The k-step: Kernel Estimation via L-BFGS-B with Downsampling 7
6.2 The $\mathbf{u}$-step: Image Recovery via FFT-ADMM 8

7 Results **9**

8 Discussion **9**

9 Limitations **9**

10 Conclusion **10**

11 Future Work **10**

1 Introduction

When taking photos, camera shake, missed focus, or improper settings can result in blurry and noisy images. Post-processing enhancement techniques aim to recover the latent sharp image from such degraded observations, by balancing contrast restoration against noise suppression. Manually adjusting these properties is tedious and subjective, so instead we formulate the problem mathematically and let an optimization algorithm find the best balance automatically.

The image degradation model we adopt is:

$$\mathbf{f} = k * \mathbf{u} + \boldsymbol{\eta}$$

where $\mathbf{f}$ is the observed blurry image, $\mathbf{u}$ is the unknown clean image, k is the unknown blur kernel (the point spread function of the camera), $*$ denotes convolution, and $\boldsymbol{\eta}$ is additive noise. The goal is to recover both $\mathbf{u}$ and k simultaneously from $\mathbf{f}$ alone, a task known as *blind image deconvolution*.

2 Problem Statement

We aim to recover the clean image $\mathbf{u}$ and blur kernel k from the degraded observation $\mathbf{f}$ by minimizing an objective function that simultaneously penalizes: (1) deviation of the reconstructed blurry image $k * \mathbf{u}$ from the observation $\mathbf{f}$, (2) excessive roughness in $\mathbf{u}$, and (3) an overly complicated or diffuse blur kernel k.

The tension between objectives (1) and (2) is central to the problem. Minimizing the data fidelity term alone would tend to fit $\mathbf{u}$ to the noise in $\mathbf{f}$, producing an image that matches the observation but is actually noisier than the truth. The regularization terms (2) and (3) impose prior beliefs about what a "natural" image and a "physically reasonable" blur kernel should look like, preventing the optimizer from overfitting to noise.

2.1 Decision Variables

The two unknowns we optimize over are:

- $\mathbf{u} \in R^{m \times n}$: The estimated clean image matrix, where $m \times n$ is the image resolution. Each entry represents a pixel intensity.
- $k \in R^{s \times s}$: The blur kernel (point spread function), a small $s \times s$ matrix describing how each true pixel spreads into a blurred region in the observation. The kernel is unknown and must be recovered alongside the image.

2.2 Objective Function

The full objective to minimize over k and $\mathbf{u}$ is:

$$\min_{k,\,\mathbf{u}} \underbrace{\|k * \mathbf{u} - \mathbf{f}\|_2^2}_{\text{data fidelity}} + \underbrace{\lambda_1 \|\nabla \mathbf{u}\|_1}_{\text{TV regularization}} + \underbrace{\lambda_2 \|D\mathbf{u}\|_2^2}_{\text{Tikhonov regularization}} + \underbrace{\mu \|k\|_1}_{\text{kernel sparsity}}$$

Each term plays a distinct and complementary role:

- **Data fidelity** $\|k * \mathbf{u} - \mathbf{f}\|_2^2$: This squared ℓ_2 term measures how well the reconstructed blurry image $k*\mathbf{u}$ matches the observed image $\mathbf{f}$. Minimizing this alone is uselss because any combination of k and $\mathbf{u}$ whose convolution equals $\mathbf{f}$ is equally optimal, so the model will just output the blurry image.
- **Total Variation (TV) regularization** $\lambda_1 \|\nabla \mathbf{u}\|_1$: The TV norm sums the magnitude of the image gradient at every pixel. Because it uses an ℓ_1 norm over the gradient, it imposes a *linear* penalty on gradient magnitude. This means large and small gradients are penalized at the same per-unit rate, so the optimizer is willing to allow a small number of pixels with large gradients (i.e., sharp edges) as long as the image is piecewise flat elsewhere. TV regularization is therefore well-suited for preserving sharp edges while enforcing uniformity in smooth regions.
- **Tikhonov regularization** $\lambda_2 \|D\mathbf{u}\|_2^2$: Here D is a discrete Laplacian or gradient operator, and the penalty is a *quadratic* (squared ℓ_2) norm of the image gradients. Because this penalty grows as the square of the gradient magnitude, even moderately large gradients incur a disproportionately high cost. The optimizer therefore suppresses gradients everywhere rather than concentrating them at a few edges. This encourages overall spatial smoothness and penalizes fine-grained oscillations and texture noise. Compared to TV, Tikhonov acts more like a blur: it averages out rapid intensity changes uniformly across the image.
- **Kernel sparsity** $\mu \|k\|_1$: The ℓ_1 norm on the kernel promotes sparsity, meaning most kernel entries are pushed toward zero. This reflects the physical expectation that the blur should be localized – only a small spatial neighborhood contributes to each blurred pixel – preventing the estimated kernel from spreading diffusely across the image and stabilizing the blind deconvolution process.

Why both TV and Tikhonov?

Using TV regularization alone tends to produce "staircase" artifacts: the piecewise-constant prior it imposes can create blocky, artificial-looking flat regions even in areas of the image that should have smooth gradients. Tikhonov regularization, applied alongside TV, counteracts this by penalizing abrupt transitions in smooth regions. The combination thus achieves the best of both: sharp edges where the image genuinely changes, and smooth gradients in uniform areas.

2.3 Constraints

The following constraints encode physical and representational requirements:

- $k \geq 0$ (entry-wise): Blur kernel entries must be non-negative. Negative kernel entries would correspond to a camera that "subtracts" light from certain directions, which has no physical interpretation.
- $\mathbf{1}^T k = 1$ (summing over all entries): The kernel must sum to one. This is a conservation-of-energy constraint: a blur redistributes light but does not create or destroy it, so the total intensity of the blurred image should equal that of the sharp image.
- $0 \leq \mathbf{u} \leq 255$ (entry-wise): Pixel intensities must lie within the valid range for an 8-bit grayscale image.

3 Mathematical Tools and Theory

The formulation above brings together several key mathematical concepts, each handling a different aspect of the problem.

3.1 Total Variation Norm

The discrete isotropic TV norm of an image $\mathbf{u}$ is defined as:

$$\|\nabla \mathbf{u}\|_1 = \sum_{i,j} \sqrt{(D_x \mathbf{u})_{ij}^2 + (D_y \mathbf{u})_{ij}^2}$$

where D_x and D_y are forward finite-difference operators in the horizontal and vertical directions respectively. The isotropic formulation takes the ℓ_2 norm of the gradient vector at each pixel before summing, making it rotationally symmetric and better at preserving diagonal edges than the anisotropic variant.

3.2 Tikhonov Regularization

The Tikhonov term uses the discrete Laplacian D to measure image roughness:

$$\|D\mathbf{u}\|_2^2 = \sum_{i,j} \left[(D_x\mathbf{u})_{ij}^2 + (D_y\mathbf{u})_{ij}^2 \right]$$

This is simply the squared ℓ_2 norm of the entire gradient field, and it corresponds to minimizing the total squared gradient energy of the image. Because this is a strictly convex quadratic in $\mathbf{u}$, it has excellent analytical properties and its gradient is straightforward to compute, which simplifies solver design.

3.3 Fast Fourier Transform (FFT) for Convolution

The convolution $k * \mathbf{u}$ appearing in the data fidelity term is expensive to compute directly. Instead, the convolution theorem allows us to compute circular convolutions in the frequency domain:

$$k * \mathbf{u} = \mathcal{F}^{-1}(\mathcal{F}(k) \odot \mathcal{F}(\mathbf{u}))$$

where $\mathcal{F}$ is the 2D discrete Fourier transform and $\odot$ is element-wise multiplication. Using the FFT algorithm, each transform costs $O(mn \log(mn))$, making frequency-domain convolution dramatically faster for large images. This also allows us to compute the normal equations for the $\mathbf{u}$-subproblem (see Section 6) in closed form entirely in the frequency domain.

3.4 Alternating Direction Method of Multipliers (ADMM)

ADMM is a convex optimization algorithm designed for problems of the form:

$$\min_{x,\, z} f(x) + g(z) \quad \text{subject to} \quad Ax + Bz = c$$

It introduces an augmented Lagrangian and alternates between minimizing over x, minimizing over z, and updating the dual variable. The key advantage over monolithic solvers is that each subproblem can often be solved in closed form via a proximal operator, even when the global problem has no such solution.

In our setting, ADMM is applied to the $\mathbf{u}$-subproblem by introducing auxiliary variables $\mathbf{v}_x = D_x\mathbf{u}$ and $\mathbf{v}_y = D_y\mathbf{u}$. This variable splitting separates the non-differentiable TV norm (now a function of $\mathbf{v}_x, \mathbf{v}_y$ alone) from the smooth quadratic terms (functions of $\mathbf{u}$ alone). The $\mathbf{u}$-update becomes a linear system that can be solved via FFT, and the $\mathbf{v}$-updates correspond to isotropic soft-thresholding, computed entry-wise in closed form.

3.5 L-BFGS-B for Kernel Estimation

L-BFGS-B (Limited-memory Broyden–Fletcher–Goldfarb–Shanno with Bounds) is a quasi-Newton method that approximates the inverse Hessian using a small

number of stored gradient vectors, making it memory-efficient for large-scale smooth optimization. We use it for the k-subproblem because the objective is smooth (with a subgradient used for the ℓ_1 sparsity term) and k must satisfy simple bound constraints $k \geq 0$. After the L-BFGS-B step, the kernel is projected onto the probability simplex to enforce $\mathbf{1}^T k = 1$.

4 Why a Direct Solver is Infeasible

A natural first approach is to hand the full problem to a general-purpose convex solver such as CVXPY, which automatically translates a high-level problem description into a standard conic form and solves it. This works well for small problems, and we indeed prototyped and validated our formulation using CVXPY on small synthetic images.

However, this approach breaks down entirely for realistic image sizes. The data fidelity term $\|k*\mathbf{u}-\mathbf{f}\|_2^2$ involves the convolution operator, which CVXPY would need to represent as a matrix multiplication $H_k\mathbf{u}$, where H_k is the $mn \times mn$ circulant matrix associated with the kernel k. For an image of even modest resolution, say 512×512, this matrix has dimension 262144×262144, containing roughly 6.87×10^{10} entries. Storing this matrix in double precision would require approximately **549 GB of memory**, far exceeding what is available on any standard machine. Even sparse representations offer little relief because the circulant matrix from a spatially extended kernel has many nonzero diagonals.

This memory bottleneck motivates abandoning the general-purpose solver entirely and instead exploiting the specific algebraic structure of the problem: namely, that convolution is diagonalized in the Fourier domain and can be evaluated in $O(mn\log(mn))$ time and $O(mn)$ memory. The FFT-ADMM algorithm described in Section 6 does exactly this.

5 Handling the Bilinear Objective: Alternating Minimization

Even once we adopt FFT-based convolution, the joint objective in k and $\mathbf{u}$ remains non-convex. The data fidelity term $\|k * \mathbf{u} - \mathbf{f}\|_2^2$ is *bilinear* in the pair $(k, \mathbf{u})$: it is linear in k when $\mathbf{u}$ is fixed, and linear in $\mathbf{u}$ when k is fixed, but it is a product of two unknowns and therefore nonlinear (and non-convex) when both are free simultaneously. Consequently, jointly minimizing over $(k, \mathbf{u})$ is a non-convex problem with no global optimality guarantee if tackled directly.

In this problem, we use *alternating minimization*: fix one variable and minimize over the other, then swap roles and repeat. Formally, with t denoting the iteration count:

1. **u-step:** Fix $k^{(t)}$ and solve $\mathbf{u}^{(t+1)} = \arg\min_{\mathbf{u}} \|k^{(t)} * \mathbf{u} - \mathbf{f}\|_2^2 + \lambda_1 \|\nabla \mathbf{u}\|_1 + \lambda_2 \|D\mathbf{u}\|_2^2$, subject to $0 \leq \mathbf{u} \leq 255$.
2. **k-step:** Fix $\mathbf{u}^{(t+1)}$ and solve $k^{(t+1)} = \arg\min_{k} \|k * \mathbf{u}^{(t+1)} - \mathbf{f}\|_2^2 + \mu \|k\|_1$, subject to $k \geq 0$ and $\mathbf{1}^T k = 1$.

With one of the two variables fixed, each subproblem becomes *strictly convex*: the bilinear term collapses to a quadratic in the free variable, and all remaining terms are convex. This guarantees that each step decreases the objective, and that each subproblem can be solved to global optimality.

Alternating minimization does not guarantee convergence to a global minimizer of the joint problem (due to non-convexity), but it is still better nothing.

6 Modelling Process

The full algorithm is summarized in Algorithm 1. The two subproblems require quite different strategies, described below.

Algorithm 1 Alternating Minimization for Blind Image Deconvolution

```
Initialize k^(0) as a small Gaussian kernel; initialize u^(0) ← f
for t = 0, 1, ..., T − 1 do
    Downsample and normalize f and u^(t) to 64 × 64 and [0, 1]
    Solve k-step via L-BFGS-B with bounds k ≥ 0 on downsampled problem
    Project k onto the probability simplex (1^T k = 1)
    Upsample k^(t+1) back to original scale
    Solve u-step via FFT-ADMM at full resolution with fixed k^(t+1)
    u^(t+1) ← clip(u^(t+1), 0, 255)
end for
return u^(T), k^(T)
```

6.1 The k-step: Kernel Estimation via L-BFGS-B with Downsampling

With $\mathbf{u}$ fixed, the k-subproblem is:

$$\min_{k \geq 0,\ \mathbf{1}^T k = 1} \|k * \mathbf{u} - \mathbf{f}\|_2^2 + \mu \|k\|_1$$

Although this is convex in k, solving it at the full image resolution is unnecessarily expensive: the kernel estimation only needs to capture the coarse structure of the blur, and the fine image details do not substantially alter this estimate. We therefore downsample both $\mathbf{f}$ and $\mathbf{u}$ to 64×64 pixels and normalize pixel intensities to $[0, 1]$ before running the kernel solver.

This downsampling is the single most impactful design decision for solver speed. At full resolution, each objective evaluation requires a convolution costing $O(mn \log(mn))$

for an $m \times n$ image; at 64×64, this is reduced by a factor of $\frac{mn \log(mn)}{64^2 \log(64^2)}$, which for a 512×512 image is a speedup of roughly $64\times$. In practice this drops k-step wall-clock time to approximately 60ms per outer iteration while preserving the accuracy of the recovered kernel, since blur is a low-frequency phenomenon and is well-represented at reduced resolution.

The ℓ_1 sparsity term $\mu\|k\|_1$ is non-differentiable at zero, so we use a subgradient in place of an exact gradient when any kernel entries are at or near zero. L-BFGS-B handles the non-negativity constraint $k \geq 0$ via projected gradient steps. After convergence, the kernel is projected onto the simplex via an $O(s^2 \log s^2)$ sorting-based algorithm to enforce $\mathbf{1}^T k = 1$.

6.2 The u-step: Image Recovery via FFT-ADMM

With k fixed, the $\mathbf{u}$-subproblem is:

$$\min_{0 \leq \mathbf{u} \leq 255} \|k * \mathbf{u} - \mathbf{f}\|_2^2 + \lambda_1 \|\nabla \mathbf{u}\|_1 + \lambda_2 \|D\mathbf{u}\|_2^2$$

This is a strictly convex problem in $\mathbf{u}$, but it cannot be solved in closed form because of the non-differentiable TV term $\|\nabla \mathbf{u}\|_1$. We apply ADMM with variable splitting: introduce auxiliary variables $\mathbf{v}_x = D_x \mathbf{u}$ and $\mathbf{v}_y = D_y \mathbf{u}$, and rewrite the problem as:

$$\min_{\mathbf{u},\, \mathbf{v}_x,\, \mathbf{v}_y} \|k * \mathbf{u} - \mathbf{f}\|_2^2 + \lambda_1 \sum_{i,j} \sqrt{(\mathbf{v}_x)_{ij}^2 + (\mathbf{v}_y)_{ij}^2} + \lambda_2 \|\mathbf{v}_x\|_2^2 + \lambda_2 \|\mathbf{v}_y\|_2^2$$

subject to $\mathbf{v}_x = D_x \mathbf{u}$ and $\mathbf{v}_y = D_y \mathbf{u}$.

The augmented Lagrangian introduces dual variables $\boldsymbol{\alpha}_x$, $\boldsymbol{\alpha}_y$ and a penalty parameter ρ. Each ADMM iteration consists of three closed-form steps:

1. **u-update:** The optimality condition for $\mathbf{u}$ yields a linear system:

   $$\left(\hat{H}_k^* \odot \hat{H}_k + \rho|\hat{D}_x|^2 + \rho|\hat{D}_y|^2\right) \hat{\mathbf{u}} = \hat{H}_k^* \hat{\mathbf{f}} + \rho \hat{D}_x^* (\widehat{\mathbf{v}_x - \boldsymbol{\alpha}_x}) + \rho \hat{D}_y^* (\widehat{\mathbf{v}_y - \boldsymbol{\alpha}_y})$$

   where hats denote Fourier transforms. This is solved pointwise in the frequency domain in $O(mn \log(mn))$ via FFT, then transformed back.

2. **v-update:** Combining the TV and Tikhonov terms, the $\mathbf{v}$-update at each pixel (i, j) is:

   $$(\mathbf{v}_x, \mathbf{v}_y)_{ij} = \frac{\max\left(\|(r_x, r_y)_{ij}\| - \frac{\lambda_1}{2\lambda_2 + \rho}, 0\right)}{\|(r_x, r_y)_{ij}\|} \cdot (r_x, r_y)_{ij}$$

   where $(r_x)_{ij} = (D_x \mathbf{u} + \boldsymbol{\alpha}_x)_{ij}$ and similarly for r_y. This is isotropic soft-thresholding with a threshold shifted by the Tikhonov contribution, computed entry-wise in $O(mn)$.

3. **Dual update:** $\boldsymbol{\alpha}_x \leftarrow \boldsymbol{\alpha}_x + D_x \mathbf{u} - \mathbf{v}_x$, and similarly for $\boldsymbol{\alpha}_y$.

Pixel bounds are enforced by clipping $\mathbf{u}$ to $[0, 255]$ after each $\mathbf{u}$-update.

7 Results

Using the alternating minimization approach, the algorithm recovers an estimated clean image $\mathbf{u}$ and blur kernel k from the degraded observation $\mathbf{f}$. The hyperparameter λ_1 controls edge sharpness: increasing it allows the TV term to preserve stronger gradients, resulting in crisper edges. The hyperparameter λ_2 controls overall spatial smoothness: increasing it suppresses fine-grained noise and texture by more aggressively penalizing gradient energy. The parameter μ controls kernel compactness: it prevents the estimated blur from spreading beyond its true spatial extent and stabilizes the alternating minimization.

In our experiments, the two-step algorithm successfully suppressed blur and additive noise while preserving perceptually important edges. The downsampled kernel estimation remained accurate, and FFT-ADMM converged reliably within a moderate number of inner iterations.

8 Discussion

The joint use of TV and Tikhonov regularization reflects the complementary nature of ℓ_1 and ℓ_2 penalties on image gradients. TV's ℓ_1 character makes it indifferent between many small gradients and a few large ones so it penalizes each unit of gradient magnitude equally, so concentrating gradients at edges is no worse than spreading them out. Tikhonov's ℓ_2 character, on the other hand, hugely penalizes large gradients due to quadratic form, so it actively discourages sharp transitions everywhere except where TV explicitly allows them. Together they produce images that are smooth in uniform regions and sharp at genuine boundaries.

The decision to downsample the image for the kernel estimation step was critical. A 512×512 image processed at full resolution would require evaluating convolutions involving roughly 2.6×10^5 pixels per outer iteration; at 64×64 the workload is 4096 pixels, a $64\times$ reduction. Since blur is inherently a low-frequency phenomenon, the kernel estimated at reduced resolution transfers faithfully to the full-resolution $\mathbf{u}$-step.

9 Limitations

Several limitations of the current formulation are worth noting:

- **Grayscale images only.** The model operates on single-channel intensity matrices. Extending it to RGB images requires processing three channels.
- **Suboptimal hyperparameters.** The regularization weights λ_1, λ_2, and μ are set manually and are not guaranteed to be optimal. Different images, blur types, or noise levels may benefit from different hyperparameter values.

- **Local convergence only.** As discussed above, alternating minimization on the non-convex joint problem does not guarantee global optimality. The recovered solution depends on the initialization and may differ across runs if any randomness is introduced.

10 Conclusion

We formulated blind image deconvolution as a joint minimization of a data fidelity term and two complementary regularizers – Total Variation for edge preservation and Tikhonov for smoothness – plus a sparsity penalty on the blur kernel. Because the joint objective is bilinear and non-convex, we decomposed it into two alternating convex subproblems. The k-step uses L-BFGS-B on a downsampled problem for computational efficiency, and the $\mathbf{u}$-step uses FFT-ADMM to exploit the Fourier structure of the convolution operator and achieve $O(mn \log(mn))$ complexity per iteration. Together these choices make the algorithm practical for real images, avoiding the memory bottleneck that makes general-purpose solvers like CVXPY infeasible at full resolution.

11 Future Work

Promising directions for future work include:

- **Automatic hyperparameter tuning.** Developing a principled method like the L-curve criterion, Stein's Unbiased Risk Estimator (SURE), or cross-validation to select λ_1, λ_2, and μ automatically, eliminating manual tuning and improving generalizability across image types.
- **Color image extension.** Adapting the model to handle RGB images with a joint multi-channel Total Variation penalty to prevent color bleeding artifacts at edges.
- **Non-uniform blur kernels.** The current model assumes a spatially invariant blur kernel (i.e., the same kernel applies everywhere in the image). In practice, motion blur and defocus can vary across the image. Extending to spatially varying kernels would improve reconstruction quality for such cases.